

Computer Vision, Lezione 4

Laboratorio: segnali, frequenze, convoluzione, template matching

Appunti ricavati dai notebook in Lessons/L4/lab/ (demo, classification, 15lif)

Come leggere questi appunti

Il laboratorio fa **una sola cosa** vista da tre lati: capire un segnale guardando *di quali frequenze è fatto*, e modificarlo con la *convoluzione*.

Parte	Notebook	Domanda a cui risponde
1	demo.ipynb	Che cos'è lo spettro di un segnale e come si filtra?
2	classification.ipynb	Si può riconoscere una parola dal suo spettro?
3	15lif.ipynb	Come si cerca una forma dentro un'immagine?

Si usa l'audio perché è un segnale 1D, facile da disegnare e da ascoltare. Tutto quello che vale qui vale identico per le immagini (segnali 2D): è la base delle lezioni successive (teoria in lezione 3, aliasing in lezione 6).

Ogni sezione ha tre tipi di riquadro: **blu** = il concetto in due righe, **rosso** = punti in cui il notebook sbaglia o è ambiguo, **verde** = cosa ricordare.

Tre parole da sapere prima di iniziare

- **Frequenza di campionamento (SR)**: quanti numeri al secondo descrivono il suono. Qui SR = 22050, cioè 22050 campioni al secondo.
- **Spettro**: il grafico che dice *quanta* di ogni frequenza c'è nel segnale. Si ottiene con la FFT (Fast Fourier Transform, l'algoritmo veloce per la trasformata di Fourier).
- **Kernel**: un piccolo vettore (o matrice, nelle immagini) che si fa scorrere sul segnale. A ogni posizione si moltiplicano i valori sovrapposti e si sommano. Questa operazione è la convoluzione.

1 Segnali e frequenze (demo.ipynb)

1.1 Il segnale di partenza: un accordo

Si costruisce un accordo di La minore sommando tre sinusoidi pure: A3 = 220 Hz, C4 = 261.63 Hz, E4 = 329.63 Hz, più un po' di rumore. Durata 2 s, quindi $2 \times 22050 = 44100$ campioni.

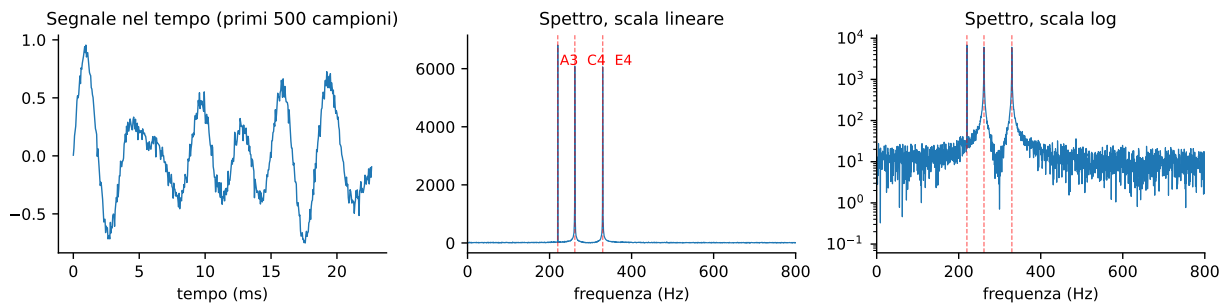


Figura 1: A sinistra il segnale nel tempo: un'onda confusa, non si vede che è fatta di tre note. Al centro e a destra lo spettro: tre picchi netti, esattamente sulle tre note (linee rosse). In scala log si vede anche il rumore di fondo.

L'idea in breve

Nel tempo un accordo è un'onda incomprensibile. In frequenza è **tre picchi**. La FFT passa da una vista all'altra senza perdere informazione (la IFFT torna indietro).

1.2 La FFT in NumPy: i numeri da sapere

Cosa	Formula	Qui
Lunghezza di <code>np.fft.rfft(x)</code> con N campioni	$N/2 + 1$	22051
Asse delle frequenze in Hz	<code>np.fft.rfftfreq(N, d=1/SR)</code>	0 ... 11025
Risoluzione (distanza tra due punti dello spettro)	$SR/N = 1/\text{durata}$	0.5 Hz
Frequenza massima rappresentabile (Nyquist)	$SR/2$	11025 Hz

- `rfft` = FFT per segnali reali: restituisce solo la metà utile dello spettro (l'altra metà è speculare).
- **Risoluzione**: dipende solo dalla *durata*. Per distinguere due note molto vicine serve registrare più a lungo, non campionare più fitto.
- **Nyquist**: con SR campioni al secondo non si possono rappresentare frequenze sopra $SR/2$. Ripreso nella lezione 6 (aliasing).
- **Scala log**: quasi tutti i grafici di spettri, anche delle immagini, si fanno in scala log, perché i valori variano di molti ordini di grandezza.

1.3 Filtrare in frequenza: tre passi

La funzione `freq_filter` del notebook:

1. $X = \text{np.fft.rfft}(x)$: passo allo spettro;

2. $X * \text{maschera}$: multiplico per un vettore di 0 e 1 (1 = frequenza che tengo, 0 = frequenza che butto);
3. `np.fft.irfft(...)`: torno nel tempo.

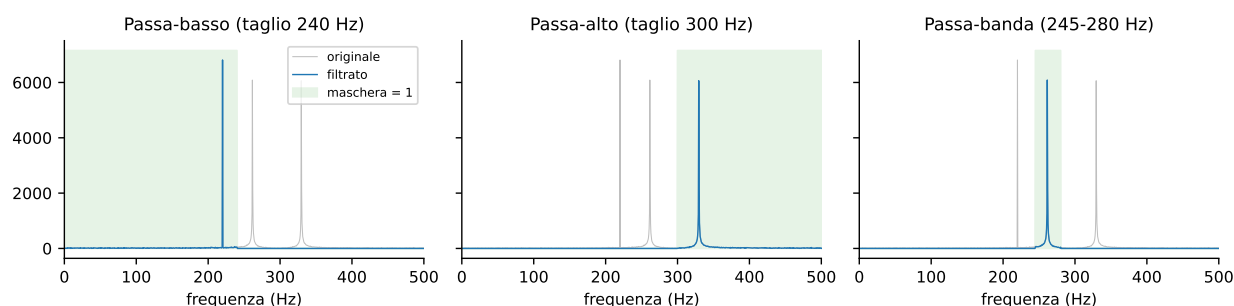


Figura 2: In grigio lo spettro originale, in blu quello filtrato, in verde la zona dove la maschera vale 1. 1. Passa-basso: resta solo A3. Passa-alto: resta E4 (più il rumore). Passa-banda: resta solo C4. Verificato: le note escluse hanno ampiezza 0 dopo il filtro.

Filtro	Cosa tiene	Effetto all'ascolto
Passa-basso	frequenze basse	suono ovattato, "attraverso il muro"
Passa-alto	frequenze alte	niente bassi (il telefono tiene circa 300-3400 Hz)
Passa-banda	una fascia	isola una sola nota

Perché una maschera 0/1 dà problemi. Un taglio netto in frequenza corrisponde, nel tempo, a una *sinc*: una funzione che oscilla a lungo prima di spegnersi. Queste oscillazioni si chiamano **ringing** ("eco" attorno ai bordi netti). Nelle immagini si vedono come aloni attorno ai contorni.

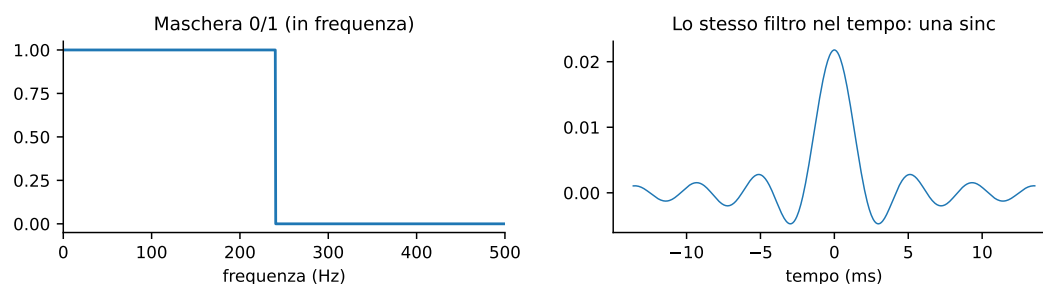
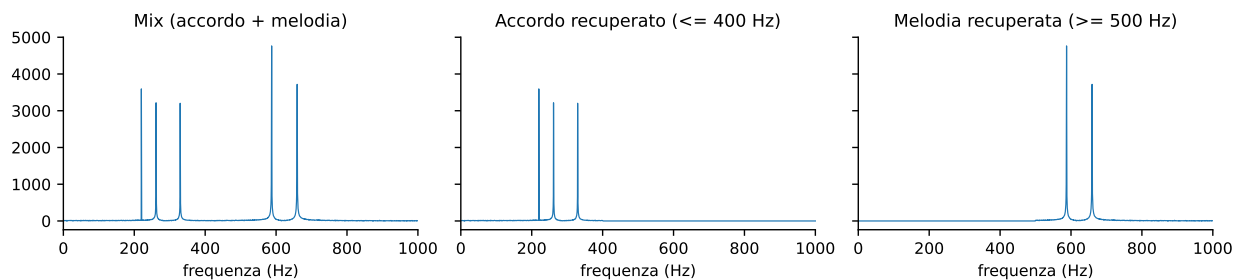


Figura 3: Lo stesso passa-basso visto nei due domini. Taglio netto a sinistra, oscillazioni lunghe a destra.

1.4 Separare due strumenti

Si somma all'accordo (tutto sotto 400 Hz) una melodia (D5 = 587 Hz, E5 = 659 Hz, sopra 500 Hz). Un passa-basso a 400 Hz recupera l'accordo, un passa-alto a 500 Hz la melodia. Funziona *solo perché le due fasce non si sovrappongono*. Con strumenti veri, che hanno armoniche su molte frequenze, la separazione è imperfetta. Il rumore dell'accordo, che sta su tutte le frequenze, finisce in parte anche nella melodia.



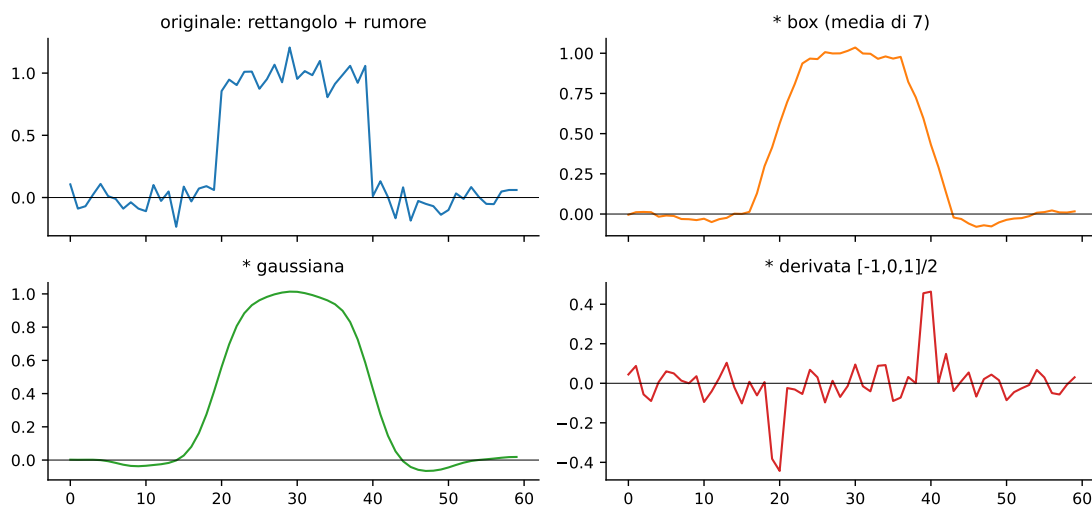
1.5 Convoluzione 1D

Cosa fa. Il kernel scorre lungo il segnale. A ogni posizione: moltiplica i valori sovrapposti, somma, scrive il risultato. `np.convolve(x, h, mode=...)` lo fa in NumPy.

Il parametro **mode** decide cosa fare ai bordi, e quindi quanto è lungo il risultato (segnale di 60 campioni, kernel di 7, valori stampati dal notebook):

mode	Lunghezza	Qui	Significato
full	$N + K - 1$	66	ogni posizione in cui kernel e segnale si toccano
same	N	60	stessa lunghezza dell'ingresso (il più comodo)
valid	$N - K + 1$	54	solo dove il kernel sta tutto dentro il segnale

I tre kernel fondamentali, applicati a un rettangolo con rumore:



Kernel	Cosa fa nel tempo	Cosa fa in frequenza
Box $[1, 1, 1, 1, 1, 1, 1]/7$	media: sfoca, toglie il rumore	passa-basso, ma con "lobi" (ringing)
Gaussiana (15 campioni, std 2.5)	sfocatura morbida	passa-basso pulito
Derivata $[-1, 0, 1]/2$	evidenzia i salti (bordi), amplifica il rumore	vedi riquadro rosso sotto

1.6 Il teorema di convoluzione: perché tutto torna

$$\mathcal{F}\{x * h\} = \mathcal{F}\{x\} \cdot \mathcal{F}\{h\}$$

Tradotto: **convolvere nel tempo = moltiplicare in frequenza**. Quindi la FFT del kernel, chiamata *risposta in frequenza* $H(f)$, dice subito quali frequenze il kernel lascia passare (dove $|H|$ è vicino a 1) e quali elimina (dove $|H|$ è vicino a 0). Il filtro con maschera 0/1 di prima è lo stesso concetto fatto direttamente in frequenza.

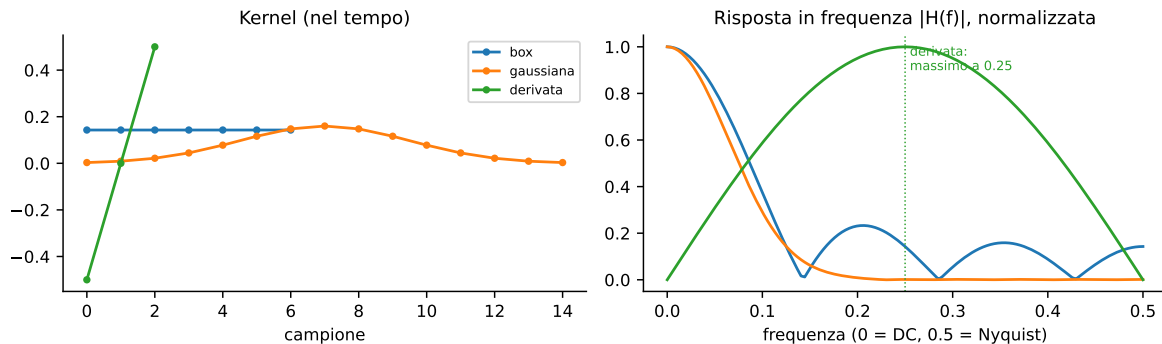


Figura 4: Frequenza in cicli per campione: 0 = componente costante (DC), 0.5 = Nyquist.

Kernel	$ H $ a Nyquist (DC = 1)	Tipo di filtro
Box	0.143	passa-basso che lascia passare un po' di alte
Gaussiana	0.0015	passa-basso che le toglie quasi tutte
Derivata	0	zero a DC e a Nyquist, massimo a 0.25

Attenzione: il notebook sbaglia o confonde

- 1. Il segno della derivata.** `np.convolve` *ribalta* il kernel prima di farlo scorrere. Con $[-1, 0, 1]/2$ il fronte di salita del rettangolo dà -0.5 (non $+0.5$) e quello di discesa $+0.5$. Con `np.correlate`, che non ribalta, la salita dà $+0.5$. Verificato su un rettangolo senza rumore.
- 2. “La derivata è un passa-alto”.** Il notebook dice che la risposta “cresce verso Nyquist”. Non è vero: $|H(f)| = |\sin(2\pi f)|$, che vale 0 in DC, 1 a $f = 0.25$ e di nuovo 0 a Nyquist. Si comporta da passa-alto solo fino a 0.25; nel complesso è un *passa-banda*. Verificato calcolando la FFT del kernel.
- 3. Refusi.** Un commento dice “band-pass” per filtri che sono passa-basso e passa-alto; un titolo dice “Band-pass filte”.

Da ricordare per l'esame

- Risoluzione della FFT = $SR/N = 1/\text{durata}$.
- Taglio netto in frequenza $\Rightarrow$ sinc nel tempo $\Rightarrow$ ringing.
- Convoluzione nel tempo = prodotto in frequenza. La FFT del kernel dice che filtro è.
- Box e gaussiana: passa-basso (la gaussiana senza ringing). Derivata: toglie la parte costante, esalta i bordi.

2 Riconoscere cifre parlate dallo spettro (classification.ipynb)

L'idea in breve

Parole diverse contengono vocali e consonanti diverse, che occupano frequenze diverse. Quindi lo spettro di una registrazione dice qualcosa su *quale* parola è stata detta. Il notebook lo usa per riconoscere le cifre da 0 a 9 e poi mostra la trappola: il classificatore impara più la *voce* della persona che la *parola*.

2.1 I dati

- **Free Spoken Digit Dataset:** 3000 registrazioni = 10 cifre $\times$ 6 persone $\times$ 50 ripetizioni, a 8000 campioni al secondo.
- Ogni clip viene portata a esattamente 1 s (8000 campioni): si aggiungono zeri se è corta, si taglia se è lunga. La FFT dà quindi 4001 valori per clip.
- **Il dataset non è nella cartella del corso:** va scaricato da GitHub:
`github.com/Jakobovski/free-spoken-digit-dataset`
e messo in
`Lessons/L4/data/free-spoken-digit-dataset-master/recordings`
Per questo i numeri qui sotto sono quelli *salvati nel notebook*, non rieseguiti.

2.2 Tre numeri che riassumono uno spettro

Invece di usare tutti i 4001 valori, si calcolano tre “riassunti”. Misurano tutti quanto il suono è acuto o “brillante”.

Feature	Cosa misura	Valori osservati
Centroide spettrale	il “baricentro” dello spettro: media delle frequenze pesata con la loro ampiezza	572-2218 Hz
Rolloff (85%)	la frequenza sotto cui sta l’85% dello spettro	717-3669 Hz
Zero-crossing rate (ZCR)	quanto spesso il segnale cambia segno (misurato nel tempo, non in frequenza)	0.014-0.425

2.3 Il classificatore: k-NN

k-NN (k nearest neighbours, $k = 5$): per classificare una clip nuova si cercano le 5 clip del training più vicine (distanza euclidea tra i vettori di feature) e si sceglie la cifra più frequente tra loro.

Prima si **normalizzano** le feature (media 0, deviazione standard 1). Serve perché il centroide vale migliaia, lo ZCR meno di 1: senza normalizzare la distanza dipenderebbe solo dal centroide.

Feature usate (split casuale 80/20)	Train	Test
3 feature	63.8%	49.8%
3 feature + tutto il log-spettro (4004 numeri)	92.9%	88.0%
Tirare a caso	10%	

- Log-spettro = $\text{np. log}(|\text{FFT}| + 1)$. Il +1 evita $\log(0) = -\infty$ dove lo spettro è nullo.
- Dopo la normalizzazione le 3 feature “pesano” come ognuna delle altre 4001 colonne, cioè quasi niente.

2.4 Il punto chiave: una persona mai sentita

Con lo split casuale, per ogni clip di test ci sono nel training decine di registrazioni *della stessa persona che dice la stessa cifra*. Il test è troppo facile.

La prova onesta è il **leave-one-speaker-out**: si allena su 5 persone e si testa sulla sesta, ripetendo per tutte e 6.

Feature	Split casuale	Persona esclusa
3 feature	49.7%	28.7%
3 feature + log-spettro	88.2%	32.2%

Da 88% a 32%. Spiegazione proposta dal notebook: lo spettro di tutta la clip contiene anche la voce (altezza, timbro), il volume, il microfono e la stanza. Il k-NN usa questi indizi finché la persona è nel training; quando è esclusa non li ha più.

Attenzione: il notebook sbaglia o confonde

Rolloff: “energia” o ampiezza? Il testo parla dell’85% dell’energia, ma il codice somma l’ampiezza (`np.cumsum(mag)`), non il suo quadrato. La differenza pratica è piccola, ma la definizione è diversa.

Normalizzazione prima dello split. Media e deviazione sono calcolate su tutto il dataset *prima* di separare train e test: un po’ di informazione del test entra nel training. L’ultima cella (con `make_pipeline`) lo fa nel modo giusto.

2.5 L’esercizio proposto (obiettivo: circa 60% sulla persona esclusa)

Suggerimenti del notebook, non implementati qui:

1. **Togliere il volume.** Un volume diverso moltiplica tutto lo spettro per una costante; dopo il logaritmo diventa una costante *sommata*. Basta sottrarre la media del log-spettro.
2. **Poche bande larghe** (qualche decina, più fitte alle basse frequenze) al posto di 4001 valori, con la potenza media per banda. Così piccole differenze di tono tra persone contano meno.
3. **Tagliare il silenzio**, dividere la parte parlata in pochi pezzi in ordine e calcolare le bande per ogni pezzo. Così si conserva l’ordine dei suoni, che lo spettro dell’intera clip perde.

Gli MFCC, le feature standard del riconoscimento vocale, mettono insieme proprio queste idee.

Da ricordare per l’esame

- Uno split casuale sovrastima l’accuratezza quando la stessa persona (o sorgente) sta sia in train sia in test. Si valuta per gruppi: leave-one-speaker-out.
- Con feature di scala molto diversa, un metodo basato su distanze (k-NN) richiede la normalizzazione.

3 Convoluzione, cross-correlazione, template matching (15lif.ipynb)

Il notebook scrive a mano, con cicli `for`, le operazioni che NumPy fa già. Serve a vedere *esattamente* cosa succede. Riferimento: capitolo 15 del libro *Foundations of Computer Vision* (vision-book.mit.edu).

3.1 Le due operazioni a confronto

	Convoluzione	Cross-correlazione
Formula 1D	$\text{out}[n] = \sum_k l[k] h[n - k]$	$\text{out}[n] = \sum_k h[k] l[n + k]$
Il kernel	viene ribaltato	non viene ribaltato
Lunghezza (nel notebook)	$N + K - 1$	N
Esempio $l = [1, 2, 3, 4]$, $h = [-1, -2, -3]$	$[-1, -4, -10, -16, -17, -12]$	$[-14, -20, -11, -4]$
Uso tipico	filtrare (sfocare, derivare)	cercare un pattern

Verificato: il risultato della convoluzione coincide con `np.convolve`; i primi due valori della cross-correlazione coincidono con `np.correlate(l, h, 'valid')` (gli ultimi due usano zeri oltre il bordo destro).

L'idea in breve

Convoluzione = cross-correlazione con il kernel ribaltato. Se il kernel è simmetrico (box, gaussiana) le due operazioni danno lo stesso risultato. Se non lo è (derivata, un triangolo) no.

3.2 La prova dell'impulso

Un **impulso** è un'immagine tutta a 0 con un solo pixel a 1. Applicarci un'operazione mostra "l'impronta" dell'operazione stessa.

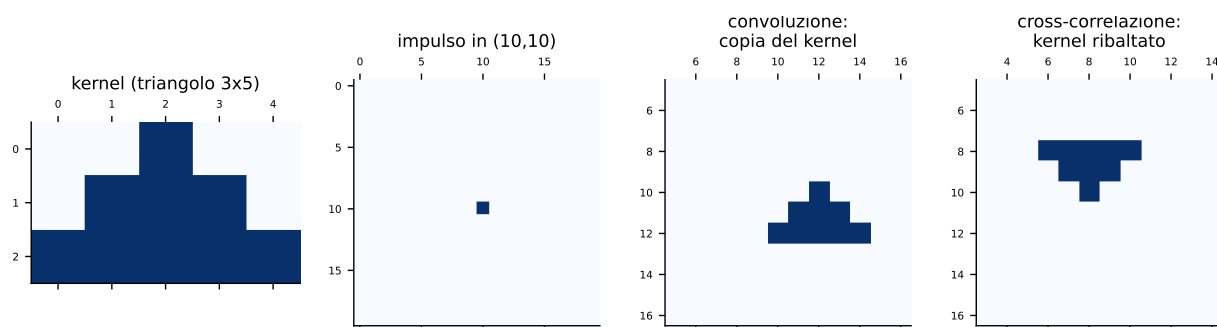


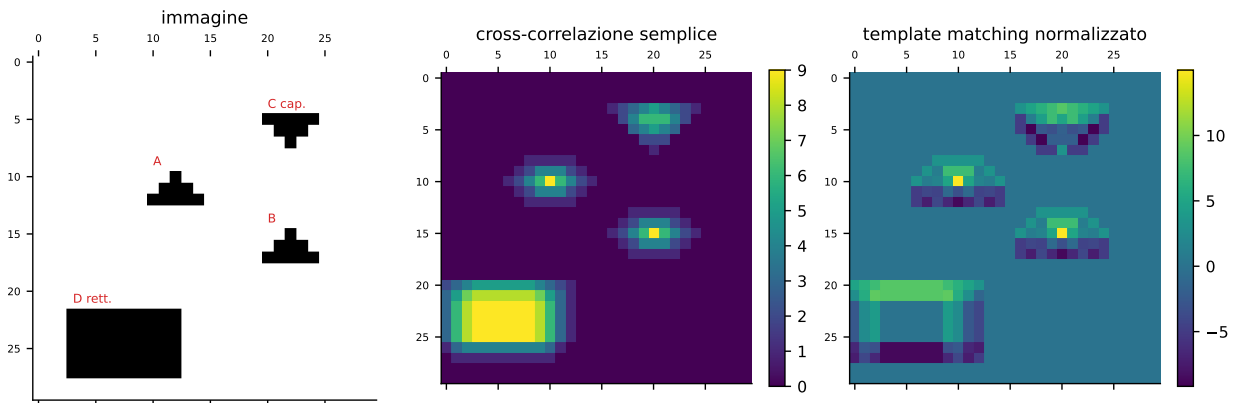
Figura 5: Impulso in (10,10). La convoluzione ricopia il kernel così com'è (righe 10-12, colonne 10-14). La cross-correlazione lo ricopia ruotato di 180 gradi (righe 8-10, colonne 6-10). Verificato eseguendo le funzioni del notebook.

3.3 Template matching: cercare una forma

Idea: faccio scorrere il pattern cercato (qui un triangolo 3×5) su tutta l'immagine e a ogni posizione misuro quanto il pezzo di immagine sotto gli somiglia. Dove il valore è massimo, lì c'è il pattern.

Questa misura è proprio la cross-correlazione.

L'immagine di prova contiene: due triangoli giusti (A e B), un triangolo capovolto (C), un rettangolo pieno di 1 (D).



Metodo	A (10,10)	B (15,20)	C capovolto (5,20)	D rettangolo
Cross-correlazione semplice	9	9	5	9
Template matching normalizzato	15.00	15.00	-1.67	0.00

Valori ottenuti eseguendo le funzioni del notebook. Il massimo cade sull'angolo in alto a sinistra del pattern, perché il kernel è "agganciato" lì.

Problema 1: la convoluzione trova la cosa sbagliata. Con la convoluzione il massimo (9) cade in (7,24), cioè sul triangolo *capovolto*, spostato di (2,4) = dimensioni del kernel meno 1. Ribaltando il kernel si cerca il pattern ribaltato. Per cercare un pattern serve la cross-correlazione.

Problema 2: la cross-correlazione semplice si fa ingannare dalla luminosità. Il rettangolo D non è un triangolo, ma prende 9 come i triangoli veri: il punteggio cresce con quanto è chiara la zona, non con quanto somiglia. È un falso positivo.

La soluzione (template_matching) normalizza in due passi:

1. il kernel viene portato a **media 0 e deviazione 1**. Su una zona uniforme (tutta chiara o tutta scura) la somma fa 0;
2. il risultato in ogni posizione viene **diviso per la deviazione standard del pezzo di immagine** sotto il kernel, cioè per il suo contrasto.

Risultato: i triangoli veri restano a 15, il rettangolo scende a 0, il capovolto diventa negativo.

Limiti (dichiarati dal notebook): funziona se il pattern si sposta, ma non se cambia *scala*, *rotazione*, *illuminazione* non uniforme, o forma tra oggetti della stessa categoria. Ai bordi il pezzo di immagine è troncato (nel codice c'è un TODO).

Da ricordare per l'esame

- Per cercare un pattern: cross-correlazione, non convoluzione.
- La cross-correlazione semplice non è normalizzata: le zone chiare danno falsi positivi. Si corregge con kernel a media 0 e divisione per il contrasto locale.
- Il template matching è robusto solo agli spostamenti.

3.4 Esercizi proposti

- Provare altri modi di gestire i bordi, per esempio il padding circolare:
`np.pad(img, p, mode='wrap')`.
- Riconoscere lettere in uno screenshot di testo usando il template di un solo carattere; provare font diversi e grassetto.
- Trovare gli uccelli nella Fig. 15.5 del visionbook con un template ritagliato a mano. Se il risultato è rumoroso, ridurre prima la risoluzione con `scipy.ndimage.zoom`.

4 Domande tipo esame

1. **Da cosa dipende la risoluzione in frequenza di una FFT?**
Vale $SR/N = 1/\text{durata}$. Dipende solo dalla durata del segnale.
2. **Perché un filtro a taglio netto in frequenza produce ringing?**
Perché nel tempo corrisponde a una sinc, che oscilla a lungo prima di spegnersi.
3. **Cosa dice il teorema di convoluzione e a cosa serve?**
Convolvere nel tempo equivale a moltiplicare in frequenza. La FFT del kernel dice quali frequenze il filtro tiene.
4. **Differenza tra convoluzione e cross-correlazione?**
Nella convoluzione il kernel è ribaltato. Per kernel simmetrici coincidono. Per cercare un pattern serve la cross-correlazione.
5. **Perché la cross-correlazione semplice sbaglia nel template matching?**
Il punteggio dipende dalla luminosità, non dalla somiglianza. Si corregge con kernel a media 0 e divisione per il contrasto locale.
6. **Perché lo split casuale sovrastima l'accuratezza sulle cifre parlate?**
La stessa persona sta sia nel train sia nel test. Con la persona esclusa l'accuratezza scende da 88.2% a 32.2%.
7. **Che filtro è $[-1, 0, 1]/2$?**
 $|H(f)| = |\sin(2\pi f)|$: zero in DC e a Nyquist, massimo a $f = 0.25$. Toglie la parte costante ed esalta i bordi.

Cosa è verificato e cosa no

- **Rieseguito**, con il codice dei notebook: spettro e filtri dell'accordo, lunghezze di `np.convolve`, risposte in frequenza dei tre kernel, segno della derivata, convoluzione e cross-correlazione 1D e 2D, prova dell'impulso, template matching. Le figure sono generate da quel codice (numpy 2.2.6, scipy 1.16.1, matplotlib 3.10.5).
- **Non rieseguito**: tutti i numeri di `classification.ipynb`, presi dagli output salvati nel notebook, perché il dataset non è presente.